

Prompting Techniques and Tool-Use Observations

Chat 1

1. One-Shot Prompting

Definition: The model is given exactly one example before being asked to perform the new task.

Input:

```
hello --> bonjour  
now translate : goodbye
```

Output:

```
Goodbye -> au revoir
```

Observation: Only a single example (hello to bonjour) was provided as a demonstration before the new query. The model correctly understood the translation pattern from just one example and applied it to a new word.

2. Few-Shot Prompting

Definition: The model is given two or more examples before being asked to perform the new task.

Input:

```
hello -> bonjour  
thanks -> merci  
translate : Goodbye
```

Output:

```
au revoir
```

Observation: Here, two examples (hello to bonjour and thanks to merci) were provided before the new input (Goodbye). With more examples, the model has a stronger pattern to understand the task, which can improve consistency of output.

3. Zero-Shot Prompting

Definition: The model is given no examples, only a direct instruction or question, and must rely entirely on its pre-trained knowledge.

Input:

if a train travels 60 km in 1.5 hours, what is its speed?

Output:

Speed = Distance / Time
60 / 1.5 = 40
Answer: 40 km/h

Observation: No examples or demonstrations were given here, just a direct question. The model relied purely on internal knowledge of the speed formula (Speed = Distance divided by Time) to solve the problem and produce a correct answer.

4. Role-Based Prompting

Definition: The model is told to act as a particular person or professional. This helps guide the response according to that role's expertise and perspective.

Input:

Act as an experienced nutritionist.
Explain why drinking enough water daily is important.

Output:

Staying properly hydrated supports nearly every function in the body. Water helps regulate body temperature, aids digestion, keeps joints lubricated, and helps flush out toxins through the kidneys. Even mild dehydration can lead to fatigue, headaches, and reduced concentration, so I always recommend clients track their intake throughout the day rather than waiting until they feel thirsty.

Observation: Once the model was asked to "act as an experienced nutritionist," the response took on a more advisory, client-facing tone, mentioning practical health effects and even offering a habit-based recommendation instead of just listing dry facts. Giving the model a role nudges it toward a more specific voice and level of expertise suited to that profession.

5. Instruction-Based Prompting

Definition: The model is given clear and specific instructions, specifying what it should do, how it should respond, and what limitations it should follow.

Input:

Explain what cloud computing is.
Use simple language, give 2 real-world examples, and answer in exactly 3 sentences.

Output:

Cloud computing means using computing power, storage, or software that runs on remote servers over the internet instead of on your own device. For example, Google Drive lets you store and access files online, and Netflix streams video directly from cloud servers to your screen. This means you can access your data or applications from anywhere, without needing powerful hardware of your own.

Observation: Several constraints were stacked here at once – simple language, exactly two examples, and exactly three sentences – and the model followed every single one precisely, keeping the explanation compact without dropping either example. This shows that the more explicit and structured the instructions, the more control there is over the exact shape of the output.

Chat 2

1. Searching Using ChatGPT's Own Tool – Chrome

Input:

what is the weather in pune

Output:

Pune right now is around 29C with a shower, and today looks like a mix of sunny spells and occasional showers.
The next few days stay pretty monsoon-y, with temperatures around 28-29C and intermittent showers/thunderstorms.
(Includes a 7-day forecast card: Sun 29/22, Mon 29/23, Tue 29/22, Wed 29/23, Thu 29/23, Fri 28/24, Sat 29/26, Sun 29/26)

Observation: The model answered directly from a live weather tool, giving real-time data (current temperature, condition, hourly trend, and a multi-day forecast). No code or function was involved – this reply was given from data fetched by an external data source (a weather tool/API called internally by the assistant).

2. Code Generation Request (Function Calling Demonstration)

Input:

write a function to get_weather(city) and shows how it would call it for pune

Output:

```
def get_weather(city):  
    # In a real program, you'd call a weather API here.  
    return f"The weather in {city} is sunny and 29C."  
  
# Call the function for Pune  
weather = get_weather("Pune")  
print(weather)
```

Result when run:

The weather in Pune is sunny and 29C.

Observation: The model correctly demonstrated the calling convention (`get_weather("Pune")`) and used Pune as the example city per the request, but explicitly noted that a real implementation would need an actual API call instead of hard-coded data. This shows the model's ability to translate a conceptual request into working, illustrative code while being transparent about its limitations.